🧠 Project Brain — Complete Technical Implementation Blueprint

> **Document Purpose:** This is the FULL technical guide for building Project Brain. Every API key, every library, every data flow, every graph structure — explained in detail. This accompanies `Project_Brain_Final_Enhanced_Idea.md` (the WHAT and WHY) with the HOW.

> **Last Updated:** February 16, 2026

TABLE OF CONTENTS

Part I: Foundation & Components

1. Complete Tech Stack & Why Each Choice
2. All API Keys & Services Setup
3. Project Directory Structure
4. Component: Intent Compiler
5. Component: Adversarial Council
6. Component: Build Engine + Anti-Cascade
7. Component: Cost Transparency Engine
8. Component: Honesty Layer

Part II: System Architecture & Infrastructure

9. Complete System Flow
10. LangGraph Implementation
11. Grandmaster Router
12. Database Schemas
13. MCP Server Implementation
14. Deployment Architecture
15. Monthly Cost Estimates

Part III: Detailed Flows, APIs & Security

16. Complete Request Lifecycle
17. Brain API Endpoints
18. Example API Request/Response
19. Memory Cortex Data Flows
20. Router Learning Over Time
21. MCP Connections
22. Error Handling & Recovery
23. Security Architecture

---

1. COMPLETE TECH STACK

Core Runtime & Language

Layer	Technology	Version	Why This Choice
Language	TypeScript	5.x	Type safety prevents bugs in Brain's own code. Every component is typed.
Runtime	Node.js	22 LTS	Async-first, massive ecosystem, runs everywhere.
Package Manager	pnpm	9.x	Faster than npm, strict dependency resolution, disk-efficient monorepo support.
Monorepo	Turborepo	2.x	Parallel builds, caching, dependency graph awareness across Brain's packages.

AI Orchestration Layer

Technology	What It Does in Brain	How We Use It
LangGraph (by LangChain)	**THE BACKBONE.** Orchestrates all agent workflows as cyclic state machines.	Every Brain workflow (Intent Compiler → Council → Build → Test → Deploy) is a LangGraph graph. Supports cycles (re-debate, retry), conditional edges (route based on confidence), and persistent checkpoints.
LangChain.js	Provides LLM abstractions, prompt templates, output parsers, tool calling	Used inside each LangGraph node to call LLMs. We use `ChatOpenAI`, `ChatAnthropic`, `ChatGoogleGenerativeAI` classes.
LangSmith	Tracing, debugging, evaluation of LLM calls	Every LLM call is traced. We can replay any conversation, debug any failure, measure latency/cost per step.

LLM Providers (Multi-Model Strategy)

Provider	Models We Use	When Brain Uses It	API Cost (Feb 2026)
Anthropic	Claude Opus 4.6, Claude Sonnet 4.5	Architecture debates, complex reasoning, code review	Opus: $15/$75 per 1M tokens (input/output). Sonnet: $3/$15
OpenAI	GPT-5.2, GPT-5.3-Codex, o4-mini	Code generation, multi-step coding, fast iteration	GPT-5.2: $10/$30. Codex: $2/$8. o4-mini: $1.10/$4.40
Google	Gemini 2.5 Pro, Gemini 3 Flash	Testing, documentation, large-context analysis (1M tokens)	Pro: $1.25/$5. Flash: $0.075/$0.30
Moonshot	Kimi K2.5	Vision-to-code (UI screenshots), multimodal tasks	$0.55/$2.20 per 1M tokens
Local/Open	Qwen3-Coder-Next (80B/3B active), DeepSeek V3.2	Verification, simple tasks, cost reduction, privacy-sensitive code	FREE (runs locally via Ollama/vLLM)

Inference Optimization

Technology	What It Does	How We Use It

Technology	What It Does	How We Use It
SGLang	High-performance LLM inference server	Runs local models (Qwen3, DeepSeek) with RadixAttention for KV-cache reuse. When Brain asks the same model about the same codebase repeatedly, RadixAttention reuses cached context → 3-5x faster, 50% less memory.
Ollama	Simple local model runner	Development/testing with local models. `ollama run qwen3-coder-next`
vLLM	Production-grade inference	PagedAttention for efficient GPU memory. Continuous batching for throughput. Used in production deployment.

Data & Storage

Technology	What It Stores	Why
PostgreSQL 16	User accounts, project metadata, audit logs, governance records, tool registry	ACID transactions, JSON support, full-text search. The source of truth.
Qdrant (Vector DB)	Memory Cortex embeddings — code patterns, failure signatures, user preferences	Purpose-built for vector similarity search. Payload filtering lets us scope searches by user/project/language. Faster than Pinecone, self-hostable.
Neo4j (Knowledge Graph)	Code dependency graphs, architecture diagrams, decision chains, failure cascades	Graph queries like "show me all functions that depend on this API" or "what decisions led to this architecture choice" are natural in Neo4j.
Redis 7	Session cache, rate limiting, real-time state during builds, pub/sub for agent coordination	Sub-millisecond reads for hot data. Pub/sub for real-time agent-to-agent communication.
MinIO (S3-compatible)	Generated code artifacts, project snapshots, formal proof files, tool packages	Self-hosted object storage. Compatible with S3 API. Stores large binary artifacts without bloating PostgreSQL.

Protocols & Communication

Protocol	What It Does	How Brain Uses It
MCP (Model Context Protocol)	Standardized interface for AI ↔ Tool communication	Brain exposes MCP servers for: file system access, terminal commands, git operations, database queries, browser automation. Any MCP-compatible client can connect to Brain. Brain also CONSUMES external MCP servers (e.g., GitHub, Jira, Slack).
A2A (Agent-to-Agent Protocol)	Standardized interface for Agent ↔ Agent communication	Brain's internal agents (Architect, Attacker, Builder, Tester) communicate via A2A. External agents (Claude Code, Codex) can also participate if the user wants to combine tools.
WebSocket	Real-time bidirectional communication	Brain ↔ User UI. Live streaming of build progress, test results, confidence updates.
gRPC	High-performance internal service communication	Between Brain's microservices (Router → Inference → Verification). Faster than REST for internal calls.

Formal Verification

Technology	What It Does	How Brain Uses It
Dafny (Microsoft)	Program verification language with built-in proofs	Brain translates critical code (payment, auth, state machines) into Dafny, proves correctness, then back-translates.
Z3 (SMT Solver)	Automated theorem prover	Used by Dafny internally. Brain also uses Z3 directly for simpler invariant checks (e.g., "this function always returns a positive number").
TLA+ Spec	System-level specification	For distributed system designs (e.g., "prove that this payment flow handles concurrent requests correctly").

Sandbox & Execution

Technology	What It Does	How Brain Uses It
WebContainers (StackBlitz)	In-browser Node.js runtime	Fast preview of generated apps. User sees live output without deploying. No server needed.
E2B (Code Interpreter)	Cloud sandbox for code execution	Runs generated code in isolated containers. Brain can execute, test, and observe output safely. Each sandbox is ephemeral — destroyed after use.
Docker	Container isolation for Survival Tests	Chaos testing (kill DB, saturate network) runs in Docker containers so it doesn't affect the host.

Frontend (Brain's UI)

Technology	What It Does
Next.js 15	Brain's dashboard/UI. Server components for fast loads. App Router.
React 19	Component library for the Brain interface.
Tailwind CSS 4	Utility-first styling. Consistent design system.
shadcn/ui	Pre-built accessible UI components (dialogs, tables, charts).
Recharts	Data visualization for Survival Test scores, confidence graphs, cost breakdowns.
Monaco Editor	Code editor in the browser (same engine as VS Code). Shows generated code with inline annotations.
Xterm.js	Terminal emulation in browser. Shows Brain's terminal output, test runs, build logs.

---

2. API KEYS & SERVICES — Complete Setup Guide

Step-by-Step: Get Every API Key Brain Needs

2.1 Anthropic (Claude) — Primary Reasoning Model

```
1. Go to: https://console.anthropic.com/
2. Sign up / Log in
3. Go to: Settings → API Keys → Create Key
4. Name it: "brain-production"
5. Copy the key (starts with "sk-ant-...")

Environment variable:
```

```
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxxxxxx

Models available:
- claude-opus-4-6-20260201     (best reasoning, expensive)
- claude-sonnet-4-5-20251022   (balanced)
- claude-haiku-4-20250301      (fast, cheap)

Usage:
import Anthropic from '@anthropic-ai/sdk';
const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

const response = await anthropic.messages.create({
  model: 'claude-opus-4-6-20260201',
  max_tokens: 8192,
  system: 'You are Brain\'s Architect agent. Your role is to design system
architecture...',
  messages: [{ role: 'user', content: userPrompt }]
});

Pricing (Feb 2026):
- Opus 4.6:   $15 / 1M input tokens,  $75 / 1M output tokens
- Sonnet 4.5: $3  / 1M input tokens,  $15 / 1M output tokens
- Haiku 4:    $0.25 / 1M input,        $1.25 / 1M output

Rate limits (default):
- Tier 1: 60 RPM, 60K tokens/min
- Tier 4: 4000 RPM, 400K tokens/min
```

2.2 OpenAI (GPT-5, Codex) — Primary Code Generation

```
1. Go to: https://platform.openai.com/
2. Sign up / Log in
3. Go to: API Keys → Create new secret key
4. Name it: "brain-production"
5. Copy the key (starts with "sk-...")

Environment variable:
OPENAI_API_KEY=sk-xxxxxxxxxxxx

Models available:
- gpt-5.2                  (frontier reasoning + coding)
- gpt-5.3-codex            (optimized for code, 25% faster)
- o4-mini                   (fast reasoning, cheap)

Usage:
import OpenAI from 'openai';
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

const response = await openai.chat.completions.create({
  model: 'gpt-5.3-codex',
```

```
  messages: [
    { role: 'system', content: 'You are Brain\'s Builder agent...' },
    { role: 'user', content: taskPrompt }
  ],
  temperature: 0.2,    // Low temperature for code (deterministic)
  max_tokens: 16384,
  tools: brainTools,   // MCP tools Brain provides
});

Pricing (Feb 2026):
- GPT-5.2:      $10 / 1M input,  $30 / 1M output
- GPT-5.3-Codex: $2 / 1M input,   $8 / 1M output
- o4-mini:      $1.10 / 1M input, $4.40 / 1M output
```

2.3 Google (Gemini) — Large Context & Testing

```
1. Go to: https://aistudio.google.com/
2. Sign up with Google account
3. Go to: Get API Key → Create API Key
4. Select a Google Cloud project (or create one)
5. Copy the key

Environment variable:
GOOGLE_API_KEY=AIzaSyxxxxxxxxxx

OR for Vertex AI (enterprise):
GOOGLE_CLOUD_PROJECT=brain-production
GOOGLE_APPLICATION_CREDENTIALS=./service-account.json

Models available:
- gemini-2.5-pro           (1M token context, strong reasoning)
- gemini-3-flash           (fast, cheap, good for testing)

Usage:
import { ChatGoogleGenerativeAI } from '@langchain/google-genai';

const gemini = new ChatGoogleGenerativeAI({
  model: 'gemini-2.5-pro',
  apiKey: process.env.GOOGLE_API_KEY,
  maxOutputTokens: 8192,
  temperature: 0.1,
});

// Gemini excels at: analyzing entire codebases (1M context)
// Brain uses it for: Survival Tests, documentation generation, code review
```

2.4 Moonshot (Kimi K2.5) — Multimodal / Vision

```
1. Go to: https://platform.moonshot.ai/
2. Sign up / Log in
```

```
3. Go to: API Keys → Create
4. Copy the key

Environment variable:
MOONSHOT_API_KEY=msk-xxxxxxxxxxxx

Usage:
// Kimi K2.5 is OpenAI-API compatible
const kimi = new OpenAI({
  apiKey: process.env.MOONSHOT_API_KEY,
  baseURL: 'https://api.moonshot.ai/v1'
});

// Brain uses Kimi for: Vision-to-code (screenshot → working UI)
const response = await kimi.chat.completions.create({
  model: 'kimi-k2.5',
  messages: [{
    role: 'user',
    content: [
      { type: 'text', text: 'Convert this UI screenshot to React + Tailwind code' },
      { type: 'image_url', image_url: { url: screenshotBase64 } }
    ]
  }]
});
```

2.5 Local Models (Qwen3-Coder-Next, DeepSeek V3.2) — FREE

```
Option A: Ollama (Easy, Development)
=========================================
1. Install: https://ollama.com/download
2. Run:
   ollama pull qwen3-coder-next         # 80B params, 3B active (MoE)
   ollama pull deepseek-v3.2             # Strong reasoning
   ollama pull glm-5                     # 77.8% SWE-Bench

3. Start server:
   ollama serve    # Runs on http://localhost:11434

Environment variable:
OLLAMA_BASE_URL=http://localhost:11434

Usage in Brain:
const local = new OpenAI({
  apiKey: 'ollama',              // Ollama doesn't need a real key
  baseURL: 'http://localhost:11434/v1'
});

const response = await local.chat.completions.create({
  model: 'qwen3-coder-next',
  messages: [{ role: 'user', content: 'Verify this function...' }],
```

```
  temperature: 0.0
});

Hardware Requirements:
- Qwen3-Coder-Next (3B active): 8GB RAM minimum, runs on CPU
- Full 80B model:               32GB RAM + GPU recommended
- DeepSeek V3.2:                48GB+ VRAM for full model


Option B: SGLang (Production, High Performance)
==========================================
1. Install:
   pip install sglang[all]

2. Launch server:
   python -m sglang.launch_server \
     --model-path Qwen/Qwen3-Coder-Next-80B-A3B \
     --tp 4 \                     # Tensor parallelism across 4 GPUs
     --enable-radix-attention \   # KV-cache reuse (key for Brain!)
     --port 30000

3. RadixAttention benefit:
   When Brain analyzes the same codebase repeatedly (e.g., during
   multi-step debugging), RadixAttention reuses the cached context
   from previous calls. This means:
   - First call: 5 seconds (full context processing)
   - Second call: 0.8 seconds (cache hit on shared prefix)
   - 3-5x faster for iterative tasks
   - 50% less GPU memory

Environment variable:
SGLANG_BASE_URL=http://localhost:30000
```

2.6 Database Services

```
PostgreSQL (Primary Database)
==========================================
Option A: Local (Development)
  docker run -d \
    --name brain-postgres \
    -e POSTGRES_PASSWORD=brain_secure_pw \
    -e POSTGRES_DB=brain \
    -p 5432:5432 \
    postgres:16

Option B: Cloud (Production) — Supabase, Neon, or AWS RDS
  DATABASE_URL=postgresql://user:password@host:5432/brain

Qdrant (Vector Database for Memory Cortex)
==========================================
```

```
Option A: Docker (Development)
  docker run -d \
    --name brain-qdrant \
    -p 6333:6333 -p 6334:6334 \
    qdrant/qdrant:latest

Option B: Qdrant Cloud (Production)
  1. Go to: https://cloud.qdrant.io/
  2. Create cluster
  3. Get API key + URL

  QDRANT_URL=https://xxxx.cloud.qdrant.io:6333
  QDRANT_API_KEY=xxxxxxxxxxxx

Neo4j (Knowledge Graph)
==========================================
Option A: Docker (Development)
  docker run -d \
    --name brain-neo4j \
    -p 7474:7474 -p 7687:7687 \
    -e NEO4J_AUTH=neo4j/brain_password \
    neo4j:5

Option B: Neo4j Aura (Cloud)
  NEO4J_URI=neo4j+s://xxxx.databases.neo4j.io
  NEO4J_USERNAME=neo4j
  NEO4J_PASSWORD=xxxxxxxxxxxx

Redis (Cache & Pub/Sub)
==========================================
  docker run -d \
    --name brain-redis \
    -p 6379:6379 \
    redis:7-alpine

  REDIS_URL=redis://localhost:6379
```

2.7 Sandbox Services

```
E2B (Cloud Code Sandbox)
==========================================
1. Go to: https://e2b.dev/
2. Sign up → Get API Key
3. Environment variable:
   E2B_API_KEY=e2b_xxxxxxxxxxxx

Usage in Brain:
import { Sandbox } from '@e2b/code-interpreter';

const sandbox = await Sandbox.create({
```

```
  apiKey: process.env.E2B_API_KEY,
  template: 'node-22'    // or 'python-3.12', 'base'
});

// Run generated code safely
const result = await sandbox.runCode('console.log("Hello from Brain!")');
console.log(result.logs.stdout);  // "Hello from Brain!"

// Install packages
await sandbox.commands.run('npm install stripe express');

// Run tests
const testResult = await sandbox.commands.run('npm test');

await sandbox.close();  // Ephemeral — everything destroyed
```

2.8 Observability

```
LangSmith (LLM Tracing)
=========================================
1. Go to: https://smith.langchain.com/
2. Sign up → Settings → API Keys → Create
3. Environment variables:
   LANGCHAIN_API_KEY=lsv2_xxxxxxxxxxxx
   LANGCHAIN_TRACING_V2=true
   LANGCHAIN_PROJECT=brain-production

This automatically traces EVERY LLM call Brain makes.
You can see: latency, cost, input/output, token usage, errors.
```

Complete `.env` File

```
# ============================================
# PROJECT BRAIN — Environment Configuration
# ============================================

# --- LLM Providers ---
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxxxxxx
OPENAI_API_KEY=sk-xxxxxxxxxxxx
GOOGLE_API_KEY=AIzaSyxxxxxxxxxx
MOONSHOT_API_KEY=msk-xxxxxxxxxxxx

# --- Local Models ---
OLLAMA_BASE_URL=http://localhost:11434
SGLANG_BASE_URL=http://localhost:30000

# --- Databases ---
DATABASE_URL=postgresql://brain:password@localhost:5432/brain
QDRANT_URL=http://localhost:6333
QDRANT_API_KEY=          # empty for local
```

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=brain_password
REDIS_URL=redis://localhost:6379

# --- Sandbox ---
E2B_API_KEY=e2b_xxxxxxxxxxxx

# --- Observability ---
LANGCHAIN_API_KEY=lsv2_xxxxxxxxxxxx
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=brain-production

# --- Storage ---
MINIO_ENDPOINT=localhost
MINIO_PORT=9000
MINIO_ACCESS_KEY=brain_storage
MINIO_SECRET_KEY=brain_storage_secret

# --- App ---
NODE_ENV=development
PORT=3000
API_PORT=4000
WS_PORT=4001

# --- Security ---
JWT_SECRET=your-256-bit-secret
ENCRYPTION_KEY=your-32-byte-encryption-key
```

---

3. PROJECT DIRECTORY STRUCTURE

```
brain/
├── apps/
│   ├── web/                          # Next.js Dashboard UI
│   │   ├── app/
│   │   │   ├── layout.tsx
│   │   │   ├── page.tsx              # Landing / Project selector
│   │   │   ├── project/[id]/
│   │   │   │   ├── page.tsx          # Project workspace
│   │   │   │   ├── build/page.tsx    # Live build view
│   │   │   │   ├── tests/page.tsx    # Test results
│   │   │   │   ├── survival/page.tsx # Survival Test dashboard
│   │   │   │   └── memory/page.tsx   # Memory explorer
│   │   │   ├── settings/page.tsx
│   │   │   └── admin/
│   │   │       ├── governance/page.tsx
│   │   │       └── audit/page.tsx
│   │   ├── components/
│   │   │   ├── chat/                 # Conversation UI (Intent Compiler)
```

```
│   │   │   ├── editor/              # Monaco code editor
│   │   │   ├── terminal/            # Xterm.js terminal
│   │   │   ├── graphs/              # Recharts visualizations
│   │   │   └── council/             # Council debate viewer
│   │   └── lib/
│   │       ├── api.ts               # API client
│   │       └── websocket.ts         # Real-time updates
│   │
│   └── api/                         # Express/Fastify API Server
│       ├── src/
│       │   ├── server.ts            # Main entry point
│       │   ├── routes/
│       │   │   ├── projects.ts
│       │   │   ├── build.ts
│       │   │   ├── memory.ts
│       │   │   └── governance.ts
│       │   ├── websocket/
│       │   │   └── handler.ts       # WebSocket for live updates
│       │   └── middleware/
│       │       ├── auth.ts
│       │       ├── rateLimit.ts
│       │       └── audit.ts         # Governance audit trail
│       └── package.json
│
├── packages/
│   ├── core/                        # Brain's Core Intelligence
│   │   ├── src/
│   │   │   ├── index.ts
│   │   │   ├── brain.ts             # Main Brain orchestrator
│   │   │   └── types.ts             # Shared TypeScript types
│   │   └── package.json
│   │
│   ├── orchestrator/                # LangGraph Workflows
│   │   ├── src/
│   │   │   ├── graphs/
│   │   │   │   ├── main.graph.ts        # Master workflow graph
│   │   │   │   ├── intent.graph.ts      # Intent Compiler flow
│   │   │   │   ├── council.graph.ts     # Adversarial Council flow
│   │   │   │   ├── build.graph.ts       # Build + Anti-Cascade flow
│   │   │   │   ├── survival.graph.ts    # Survival Test flow
│   │   │   │   └── debug.graph.ts       # 3-Strike Debug flow
│   │   │   ├── nodes/
│   │   │   │   ├── architect.node.ts
│   │   │   │   ├── attacker.node.ts
│   │   │   │   ├── verifier.node.ts
│   │   │   │   ├── builder.node.ts
│   │   │   │   ├── tester.node.ts
│   │   │   │   └── reviewer.node.ts
│   │   │   ├── state/
```

```
│   │   │   │   ├── project.state.ts      # LangGraph state schema
│   │   │   │   └── checkpoints.ts        # State persistence
│   │   │   └── edges/
│   │   │       ├── router.edge.ts        # Conditional routing logic
│   │   │       └── escalation.edge.ts    # 3-Strike escalation
│   │   └── package.json
│   │
│   ├── router/                        # Grandmaster Model Router
│   │   ├── src/
│   │   │   ├── router.ts            # Main routing logic
│   │   │   ├── providers/
│   │   │   │   ├── anthropic.ts
│   │   │   │   ├── openai.ts
│   │   │   │   ├── google.ts
│   │   │   │   ├── moonshot.ts
│   │   │   │   ├── ollama.ts         # Local models
│   │   │   │   └── sglang.ts         # SGLang inference
│   │   │   ├── scoring/
│   │   │   │   ├── costScorer.ts
│   │   │   │   ├── accuracyScorer.ts
│   │   │   │   ├── latencyScorer.ts
│   │   │   │   └── reliabilityScorer.ts
│   │   │   └── history/
│   │   │       └── routingHistory.ts # Tracks which model worked best for what
│   │   └── package.json
│   │
│   ├── memory/                        # Memory Cortex (3 Levels)
│   │   ├── src/
│   │   │   ├── personal.ts          # Level 1: User preferences
│   │   │   ├── project.ts           # Level 2: Project knowledge
│   │   │   ├── collective.ts        # Level 3: Global patterns
│   │   │   ├── embeddings.ts        # Text → Vector conversion
│   │   │   ├── qdrant.client.ts     # Vector DB operations
│   │   │   ├── neo4j.client.ts      # Knowledge Graph operations
│   │   │   └── federated.ts         # Federated learning for L3
│   │   └── package.json
│   │
│   ├── verification/                  # Vericoding Engine
│   │   ├── src/
│   │   │   ├── vericoder.ts         # Main verification orchestrator
│   │   │   ├── invariants/
│   │   │   │   ├── parser.ts        # Extract invariants from spec
│   │   │   │   ├── checker.ts       # Check code against invariants
│   │   │   │   └── prover.ts        # Formal proof generation
│   │   │   ├── dafny/
│   │   │   │   ├── translator.ts    # TS/JS → Dafny translation
│   │   │   │   ├── runner.ts        # Execute Dafny verification
│   │   │   │   └── backTranslator.ts
│   │   │   └── z3/
```

```
│   │   │       └── solver.ts        # Direct Z3 SMT queries
│   │   └── package.json
│   │
│   ├── survival/                     # Survival Test Engine
│   │   ├── src/
│   │   │   ├── survivalTest.ts      # Main test orchestrator
│   │   │   ├── checks/
│   │   │   │   ├── dependencies.ts  # Dependency health
│   │   │   │   ├── security.ts      # OWASP scan
│   │   │   │   ├── scale.ts         # Load testing
│   │   │   │   ├── chaos.ts         # Chaos engineering
│   │   │   │   ├── maintainability.ts
│   │   │   │   └── futureProof.ts
│   │   │   └── scoring/
│   │   │       └── survivalScore.ts
│   │   └── package.json
│   │
│   ├── tools/                        # Self-Evolving Tool Synthesis
│   │   ├── src/
│   │   │   ├── toolSynthesis.ts     # Generate new tools
│   │   │   ├── toolRegistry.ts      # Store & retrieve tools
│   │   │   ├── toolValidator.ts     # Validate tool safety
│   │   │   └── toolSharing.ts       # Federated tool sharing
│   │   └── package.json
│   │
│   ├── governance/                   # Governance Engine
│   │   ├── src/
│   │   │   ├── auditTrail.ts        # Log every action
│   │   │   ├── approvalWorkflow.ts  # Human approval gates
│   │   │   ├── blastRadius.ts       # Estimate change impact
│   │   │   ├── rollback.ts          # Auto-rollback on failure
│   │   │   └── compliance.ts        # Regulatory checks
│   │   └── package.json
│   │
│   ├── mcp-servers/                  # MCP Server Implementations
│   │   ├── filesystem/              # File read/write/search
│   │   ├── terminal/                # Shell command execution
│   │   ├── git/                     # Git operations
│   │   ├── browser/                 # Browser automation
│   │   ├── database/               # DB queries
│   │   └── package.json
│   │
│   └── shared/                       # Shared Utilities
│       ├── src/
│       │   ├── logger.ts
│       │   ├── errors.ts
│       │   ├── crypto.ts            # Encryption utilities
│       │   └── metrics.ts           # Performance tracking
│       └── package.json
```

```
│
├── docker/
│   ├── docker-compose.yml           # All services
│   ├── docker-compose.dev.yml       # Development overrides
│   └── Dockerfile
│
├── turbo.json                        # Turborepo configuration
├── pnpm-workspace.yaml              # Monorepo workspace
├── package.json
├── tsconfig.base.json
└── .env.example
```

---

4. COMPONENT DEEP-DIVE: INTENT COMPILER

What It Does

The Intent Compiler is Brain's FIRST point of contact. It takes a vague user request like "build me an app" and converts it into a precise, machine-verifiable **Project Specification** with formal invariants.

Why It Matters

Every AI coding tool fails here — they start coding immediately without truly understanding. Brain REFUSES to write a single line of code without a verified spec.

Internal Flowchart

```
flowchart TD
    A["🧑 User Message: Build me a ride-sharing app"] --> B

    subgraph B["📥 STEP 1: LOAD CONTEXT"]
        B1["🔍 Qdrant Personal Memory L1<br/>→ User prefers React<br/>→ User used Fireb
        B2["🔍 Qdrant Collective Memory L3<br/>→ Use WebSockets for real-time tracking<
        B3["🔍 Qdrant Failure Signatures<br/>→ GPS drift causes wrong fare calc"]
        B4["📦 Context assembled: ~2000 tokens"]
        B1 --> B4
        B2 --> B4
        B3 --> B4
    end

    B --> C

    subgraph C["❓ STEP 2: GENERATE CLARIFICATION QUESTIONS"]
        C1["🤖 LLM: Claude Sonnet 4.5 · Temp: 0.3"]
        C2["📋 System Prompt:<br/>ROLE: Brain Intent Compiler — NEVER assume<br/>CONTEX
        C3["📤 Output: 6 questions about users, payment, scale"]
        C1 --> C2 --> C3
    end
```

```
    C --> D["⏳ STEP 3: USER ANSWERS QUESTIONS<br/>Sent via WebSocket · LangGraph check

    D --> E

    subgraph E["📝 STEP 4: GENERATE SPECIFICATION"]
        E1["🤖 LLM: Claude Sonnet 4.5 · Temp: 0.1"]
        E2["📥 Input: Request + Answers + Memory Context"]
        E3["📤 Output: ProjectSpecification JSON<br/>• Features with priorities<br/>• T
        E1 --> E2 --> E3
    end

    E --> F["✅ STEP 5: USER APPROVES SPEC<br/>Feature list · Tech stack · Invariants ·
```

Example Specification Output:

```
{
  "projectName": "RideShareApp",
  "features": [
    { "id": "F1", "name": "User auth", "priority": "must", "components": ["AuthService"
    { "id": "F2", "name": "Real-time tracking", "priority": "must" }
  ],
  "techStack": {
    "frontend": "React + TypeScript",
    "backend": "Node.js + Express",
    "database": "PostgreSQL + Redis",
    "realtime": "WebSocket (Socket.io)",
    "maps": "Mapbox GL JS",
    "payments": "Stripe Connect"
  },
  "formalInvariants": [
    { "id": "INV-1", "natural": "Every ride must have exactly one driver and one rider"
    { "id": "INV-2", "natural": "Fare must always be positive and never exceed $500", "
  ],
  "estimatedCost": "$2.50",
  "estimatedTime": "45 minutes"
}
```

Prompt Engineering Details

Brain uses **structured output** parsing to ensure the LLM response is always valid JSON:

```
// How Brain forces structured output from LLMs

import { z } from 'zod';
import { ChatAnthropic } from '@langchain/anthropic';

// Define exact output schema
const SpecificationSchema = z.object({
  projectName: z.string(),
  features: z.array(z.object({
```

```
    id: z.string(),
    name: z.string(),
    priority: z.enum(['must', 'should', 'could']),
    components: z.array(z.string()),
  })),
  formalInvariants: z.array(z.object({
    id: z.string(),
    natural: z.string(),      // Human-readable
    formal: z.string(),       // Machine-checkable
    criticality: z.enum(['HIGH', 'MEDIUM', 'LOW']),
  })),
  techStack: z.record(z.string()),
  estimatedCost: z.string(),
});

// LangChain's withStructuredOutput forces valid JSON
const model = new ChatAnthropic({
  model: 'claude-sonnet-4-5-20251022',
  temperature: 0.1,
}).withStructuredOutput(SpecificationSchema);

const spec = await model.invoke(promptWithContext);
// spec is ALWAYS valid, typed, and parseable — never raw text
```

---

5. COMPONENT DEEP-DIVE: ADVERSARIAL COUNCIL

What It Does

Three AI models with DIFFERENT perspectives debate the architecture. An Architect proposes, an Attacker finds flaws, a Verifier proves or disproves the flaws using formal logic. This continues until no proven attacks remain.

Why 3 Different Models?

If all 3 agents use the same LLM (e.g., all Claude), they share the same blindspots and biases. By using Claude (Architect), GPT-5 (Attacker), and Qwen3 (Verifier), each model brings different training data, different reasoning patterns, and different failure modes.

Council Debate Example

```
sequenceDiagram
    participant ARCH as 🏗️ Architect (Claude)
    participant ATK as ⚔️ Attacker (GPT-5)
    participant VER as ✅ Verifier (Local)

    Note over ARCH, VER: CYCLE 1: Design & Attack
    ARCH->>ATK: Proposes Ride-Sharing Architecture<br/>(Postgres, Redis, WebSockets, St
    ATK->>VER: Launches 4 Attacks:<br/>1. WS Connection Drops<br/>2. Webhook Silent Fai
    VER->>ARCH: Validates Attacks:<br/>1. PROVEN (No reconnection logic)<br/>2. PROVEN
```

```
    Note over ARCH, VER: CYCLE 2: Fix & Verify
    ARCH->>ATK: Applies 4 Fixes:<br/>1. Backoff + Offline Queue<br/>2. Idempotency Keys
    ATK->>VER: Re-evaluates: No new vulnerabilities found.
    VER->>ARCH: Verifies fixes against formal invariants.
    Note over ARCH, VER: ⚖️ VERDICT: ARCHITECTURE APPROVED ✅
```

---

6. COMPONENT DEEP-DIVE: BUILD ENGINE + ANTI-CASCADE

MAD (Maximal Agentic Decomposition) — Why It Works

Example Decomposition ("Build a payment service"):

Task	Description	Lines	Status
1	TypeScript interfaces (Todo, Category)	12	✅ Passed
2	Stripe client initialization	8	✅ Passed
3	Create customer function	15	✅ Passed
4	Create payment intent	20	✅ Passed
5	Webhook handler setup	10	✅ Passed
6	Idempotency check logic	18	❌ Strike 1!
6b	Fix: Add migration + retry idempotency	22	✅ Passed
7	Error handling wrappers	15	✅ Passed
8	Wire into Express routes	25	✅ Passed

TOTAL: 145 lines · 9 tasks · 1 self-recovered strike

Hallucination Firewall — How It Actually Works

```
graph LR
    CODE["🛠️ Micro-Task Code"] --> C1["⚡ Syntax Check"]
    C1 -->|"Fail"| REJ1["❌ Parser Error"]
    C1 -->|"Pass"| C2["🧬 Type Check"]

    C2 -->|"Fail"| REJ2["❌ Type Mismatch"]
    C2 -->|"Pass"| C3["🛡️ Invariant Check"]

    C3 -->|"Fail"| REJ3["❌ INV Violation"]
    C3 -->|"Pass"| C4["🔄 Consistency Check"]

    C4 -->|"Fail"| REJ4["❌ Inconsistency"]
    C4 -->|"Pass"| C5["🕵️ Hallucination Check"]

    C5 -->|"Fail"| REJ5["❌ Fake APIs/Pkgs"]
    C5 -->|"Pass"| PASS["✅ PASSED TO SANDBOX"]
```

```
    style PASS fill:#f9f,stroke:#333,stroke-width:4px
```

---

7. COMPONENT DEEP-DIVE: COST TRANSPARENCY ENGINE

How Brain Estimates and Tracks Cost in Real-Time

```
BEFORE a build starts:
═══════════════════════

Brain estimates cost based on:
├── Project complexity (# of features, # of components)
├── Historical data (similar projects from collective memory)
├── Model pricing (current API rates)
└── Expected task count (estimated micro-tasks)

Formula:
  estimated_cost =
    (intent_compiler_tokens × claude_sonnet_price) +
    (council_cycles × 3 × avg_council_tokens × model_prices) +
    (micro_tasks × avg_task_tokens × codex_price) +
    (micro_tasks × avg_test_tokens × gemini_flash_price) +
    (verification_calls × local_model_price) +     // $0.00
    (survival_test_calls × gemini_flash_price) +
    (e2b_sandbox_minutes × e2b_price)

Example for a todo app:
  Intent: 2000 tokens × $3/1M + 1500 × $15/1M = $0.03
  Council: 2 cycles × 3 agents × ~$0.05 avg = $0.30
  Build: 15 tasks × ~$0.005 avg = $0.08
  Tests: 15 × $0.001 = $0.015
  Verification: $0.00 (local)
  Survival: $0.01
  Sandbox: 5 min × $0.005 = $0.025
  TOTAL ESTIMATE: ~$0.46

DURING a build (real-time tracking):
═══════════════════════

After EVERY LLM call:
  1. Record: model, input_tokens, output_tokens
  2. Calculate: actual_cost = tokens × model_price
  3. Update: running_total += actual_cost
  4. Send WebSocket: { type: "cost_update", current: 0.32, ceiling: 5.00 }
  5. Check: if running_total > user_ceiling × 0.9 → WARNING
  6. Check: if running_total >= user_ceiling → PAUSE build, ask user
```

```
User can set cost ceilings:
  POST /api/v1/cost/ceiling
  { "maxPerBuild": 5.00, "maxPerMonth": 100.00 }
```

---

8. COMPONENT DEEP-DIVE: HONESTY LAYER

How Brain Calculates Confidence Scores

```
Brain NEVER says "100% perfect" unless it has mathematical proof.

CONFIDENCE SCORING FOR EACH FUNCTION:

  Base score = 50 (start neutral)

  ADD points:
  +30 if Dafny/Z3 formal proof exists (VERIFIED)
  +20 if all generated tests pass
  +10 if pattern matches collective memory (>90% conf)
  +5  if code review agent finds no issues
  +5  if consistent with previous tasks

  SUBTRACT points:
  -20 if any test fails
  -15 if firewall flagged once but was fixed
  -10 if uses unfamiliar API (not in collective memory)
  -5  if complexity score is high (deep nesting, etc.)

  CATEGORIES:
  🟢 90-100: VERIFIED (formal proof OR extensive tests + pattern match)
  🟡 70-89:  HIGH CONFIDENCE (tests pass, good patterns)
  🟠 50-69:  MODERATE (some concerns, review recommended)
  🔴 0-49:   LOW CONFIDENCE (requires human review)

  Brain displays this per-function:

  | Function | Score | Status | Details |
  |----------|-------|--------|---------|
  | `addTodo()` | 🟢 96% | VERIFIED | Dafny proof exists |
  | `deleteTodo()` | 🟢 94% | VERIFIED | Formal proof exists |
  | `filterByCategory()` | 🟡 85% | HIGH | Tests pass, no formal proof |
  | `syncToStorage()` | 🟡 78% | HIGH | No formal proof for browser APIs |
  | `handleEdgeCase()` | 🟠 62% | MODERATE | Complex logic, recommend manual review |

  Brain also says WHAT IT DOESN'T KNOW:
  "I couldn't formally verify syncToStorage because
   localStorage is a browser API that doesn't have
   a formal specification. All 12 tests pass, but
```

```
I recommend manual testing on Safari (known
localStorage quirks). Confidence: 78%."
```

---

9. COMPLETE SYSTEM FLOW — How Brain Processes a Request

```
flowchart TD
    USER["🧑 USER: Build me a ride-sharing app"] --> S1

    subgraph S1["📝 STEP 1: INTENT COMPILER"]
        direction LR
        S1A["Input: Raw message + Personal Memory L1"]
        S1B["Model: Claude Sonnet 4.5 · ~$0.02"]
        S1C["Process: Load prefs → Ask questions → Generate spec → User approves"]
        S1D["Output: ProjectSpecification → PostgreSQL + Neo4j"]
        S1A --> S1B --> S1C --> S1D
    end

    S1 --> S2

    subgraph S2["⚔️ STEP 2: ADVERSARIAL COUNCIL"]
        direction LR
        S2A["🏗️ Architect — Claude Opus $0.08"]
        S2B["⚔️ Attacker — GPT-5.2 $0.05"]
        S2C["✅ Verifier — Qwen3 LOCAL $0.00"]
        S2D["Cycle until no proven attacks remain<br/>Max 5 cycles · Cost: ~$0.15-0.30"
        S2A --> S2B --> S2C --> S2D
    end

    S2 --> S3

    subgraph S3["🔨 STEP 3: BUILD ENGINE + ANTI-CASCADE"]
        direction LR
        S3A["MAD Decomposition into micro-tasks ≤30 lines"]
        S3B["Per task: Generate code GPT-5.3-Codex $0.002<br/>→ Firewall check → Tests
        S3C["3-Strike Protocol if failures"]
        S3D["Output: GeneratedProject · Cost: $0.30-2.00"]
        S3A --> S3B --> S3C --> S3D
    end

    S3 --> S4

    subgraph S4["🔬 STEP 4: VERICODING ENGINE"]
        direction LR
        S4A["Model: Qwen3 LOCAL $0.00"]
        S4B["Translate critical functions → Dafny"]
        S4C["Run Z3 Solver for formal proofs"]
        S4D["🟢 VERIFIED · 🟡 TESTED · 🔴 UNVERIFIED"]
        S4A --> S4B --> S4C --> S4D
    end
```

```
    S4 --> S5

    subgraph S5["🛡️ STEP 5: SURVIVAL TEST"]
        direction LR
        S5A["📦 Dependency Health"]
        S5B["🔒 Security OWASP"]
        S5C["📈 Scale Simulation"]
        S5D["💣 Chaos Testing"]
        S5E["🔧 Maintainability"]
        S5F["🔮 Future-Proofing"]
        S5G["Score ≥80 → PRODUCTION READY ✅<br/>Score 60-79 → Auto-fix · Score <60 → E
    end

    S5 --> S6

    subgraph S6["🧠 STEP 6: MEMORY UPDATE"]
        direction LR
        S6A["L1: Personal preferences → Qdrant"]
        S6B["L2: Project knowledge → Neo4j + Qdrant"]
        S6C["L3: Anonymized patterns → Federated Qdrant"]
    end
```

---

10. LANGGRAPH IMPLEMENTATION — The Master Workflow

How LangGraph Works in Brain

LangGraph models workflows as **directed graphs with cycles**. Each node is a function. Edges connect nodes. State flows through the graph.

```
// packages/orchestrator/src/graphs/main.graph.ts

import { StateGraph, Annotation, END } from '@langchain/langgraph';
import { PostgresSaver } from '@langchain/langgraph-checkpoint-postgres';

// ============================================
// STEP 1: Define the State Schema
// This is the "memory" that flows through the graph
// ============================================

const BrainState = Annotation.Root({
  // User input
  userMessage: Annotation<string>,
  userId: Annotation<string>,
  projectId: Annotation<string>,

  // Intent Compiler output
  specification: Annotation<ProjectSpecification | null>({ default: () => null }),
  invariants: Annotation<FormalInvariant[]>({ default: () => [] }),
  costEstimate: Annotation<CostEstimate | null>({ default: () => null }),
  userApproved: Annotation<boolean>({ default: () => false }),
```

```
  // Council output
  architecture: Annotation<VerifiedArchitecture | null>({ default: () => null }),
  attacks: Annotation<Attack[]>({ default: () => [] }),
  councilCycles: Annotation<number>({ default: () => 0 }),

  // Build output
  generatedCode: Annotation<GeneratedFile[]>({ default: () => [] }),
  microTasks: Annotation<MicroTask[]>({ default: () => [] }),
  currentTaskIndex: Annotation<number>({ default: () => 0 }),
  buildErrors: Annotation<BuildError[]>({ default: () => [] }),
  strikeCount: Annotation<number>({ default: () => 0 }),

  // Verification output
  verificationReport: Annotation<VerificationReport | null>({ default: () => null }),

  // Survival output
  survivalScore: Annotation<SurvivalReport | null>({ default: () => null }),

  // Confidence
  confidenceScores: Annotation<ConfidenceScore[]>({ default: () => [] }),

  // Governance
  auditLog: Annotation<AuditEntry[]>({ default: () => [] }),

  // Status
  phase: Annotation<'intent' | 'council' | 'build' | 'verify' | 'survival' | 'done' | '
  messages: Annotation<BaseMessage[]>({ default: () => [] }),
});

// ============================================
// STEP 2: Define the Graph Nodes
// Each node is an async function that takes state and returns partial state
// ============================================

// Node: Intent Compiler
async function intentCompilerNode(state: typeof BrainState.State) {
  const { userMessage, userId } = state;

  // 1. Load personal memory
  const personalMemory = await memoryService.getPersonal(userId);

  // 2. Generate clarification questions
  const questions = await router.route({
    task: 'intent_analysis',
    model: 'claude-sonnet-4-5',   // Balanced speed+quality
    prompt: buildIntentPrompt(userMessage, personalMemory),
  });

  // 3. Generate specification with formal invariants
  const spec = await router.route({
    task: 'spec_generation',
    model: 'claude-sonnet-4-5',
```

```
    prompt: buildSpecPrompt(userMessage, questions.answers, personalMemory),
  });

  // 4. Generate cost estimate
  const cost = await costEngine.estimate(spec);

  return {
    specification: spec,
    invariants: spec.formalInvariants,
    costEstimate: cost,
    phase: 'council' as const,
    auditLog: [...state.auditLog, {
      action: 'SPEC_GENERATED',
      timestamp: new Date(),
      data: { spec, cost },
    }],
  };
}

// Node: Adversarial Council — Architect
async function architectNode(state: typeof BrainState.State) {
  const design = await router.route({
    task: 'architecture_design',
    model: 'claude-opus-4-6',    // Best reasoning for architecture
    prompt: buildArchitectPrompt(state.specification, state.attacks),
  });
  return { architecture: design };
}

// Node: Adversarial Council — Attacker
async function attackerNode(state: typeof BrainState.State) {
  const attacks = await router.route({
    task: 'attack_generation',
    model: 'gpt-5.2',           // Different model = different perspective
    prompt: buildAttackPrompt(state.architecture),
  });
  return { attacks };
}

// Node: Adversarial Council — Verifier
async function verifierNode(state: typeof BrainState.State) {
  // Uses LOCAL model — free, no API cost
  const verified = await router.route({
    task: 'attack_verification',
    model: 'qwen3-coder-next',   // Local model, $0.00
    prompt: buildVerifierPrompt(state.attacks, state.invariants),
  });

  // Also uses Z3 SMT solver for formal checks
  const formalResults = await z3Solver.checkInvariants(
    state.invariants,
    state.attacks
  );
```

```
  return {
    attacks: verified.provenAttacks,
    councilCycles: state.councilCycles + 1,
  };
}

// Node: MAD Decomposition
async function madDecompositionNode(state: typeof BrainState.State) {
  const microTasks = await router.route({
    task: 'task_decomposition',
    model: 'claude-sonnet-4-5',
    prompt: buildMADPrompt(state.architecture),
  });
  return { microTasks, currentTaskIndex: 0, phase: 'build' as const };
}

// Node: Build Single Micro-Task
async function buildMicroTaskNode(state: typeof BrainState.State) {
  const task = state.microTasks[state.currentTaskIndex];

  // Generate code for this micro-task
  const code = await router.route({
    task: 'code_generation',
    model: 'gpt-5.3-codex',     // Fastest coder
    prompt: buildCodePrompt(task, state.generatedCode),
  });

  // HALLUCINATION FIREWALL — check before proceeding
  const firewallResult = await hallucinationFirewall.check({
    code: code.content,
    expectedType: task.expectedOutputType,
    invariants: state.invariants,
    previousCode: state.generatedCode,
  });

  if (!firewallResult.passed) {
    return {
      buildErrors: [...state.buildErrors, firewallResult.error],
      strikeCount: state.strikeCount + 1,
    };
  }

  // Generate tests
  const tests = await testGenerator.generate(code, task);

  // Run tests in sandbox
  const sandbox = await Sandbox.create({ template: 'node-22' });
  const testResult = await sandbox.runCode(tests.testCode);
  await sandbox.close();

  return {
    generatedCode: [...state.generatedCode, code],
```

```
    currentTaskIndex: state.currentTaskIndex + 1,
    confidenceScores: [...state.confidenceScores, code.confidence],
  };
}

// ==========================================
// STEP 3: Define Conditional Edges (Routing Logic)
// ==========================================

function shouldContinueCouncil(state: typeof BrainState.State): string {
  if (state.attacks.length === 0) return 'mad_decomposition';  // No attacks remain
  if (state.councilCycles >= 5) return 'escalate_human';       // Max cycles reached
  return 'architect';                                           // Re-debate
}

function shouldContinueBuild(state: typeof BrainState.State): string {
  if (state.strikeCount >= 3) return 'escalate_human';          // 3-Strike limit
  if (state.currentTaskIndex >= state.microTasks.length) return 'verify';
  return 'build_micro_task';                                    // Next task
}

function shouldRetryOrSwitch(state: typeof BrainState.State): string {
  if (state.strikeCount === 1) return 'build_micro_task';       // Retry with root caus
  if (state.strikeCount === 2) return 'switch_model_retry';     // Different model
  return 'escalate_human';                                      // Give up gracefully
}

// ==========================================
// STEP 4: Build the Graph
// ==========================================

const workflow = new StateGraph(BrainState)
  // Add nodes
  .addNode('intent_compiler', intentCompilerNode)
  .addNode('architect', architectNode)
  .addNode('attacker', attackerNode)
  .addNode('verifier', verifierNode)
  .addNode('mad_decomposition', madDecompositionNode)
  .addNode('build_micro_task', buildMicroTaskNode)
  .addNode('switch_model_retry', switchModelNode)
  .addNode('verify', vericodingNode)
  .addNode('survival_test', survivalTestNode)
  .addNode('update_memory', updateMemoryNode)
  .addNode('escalate_human', escalateToHumanNode)

  // Add edges (the flow)
  .addEdge('__start__', 'intent_compiler')
  .addEdge('intent_compiler', 'architect')
  .addEdge('architect', 'attacker')
  .addEdge('attacker', 'verifier')
  .addConditionalEdges('verifier', shouldContinueCouncil)
  .addEdge('mad_decomposition', 'build_micro_task')
  .addConditionalEdges('build_micro_task', shouldContinueBuild)
```

```
  .addConditionalEdges('switch_model_retry', shouldRetryOrSwitch)
  .addEdge('verify', 'survival_test')
  .addEdge('survival_test', 'update_memory')
  .addEdge('update_memory', '__end__');

// ============================================
// STEP 5: Compile with Checkpointing
// Checkpoints let us pause/resume the workflow
// ============================================

const checkpointer = new PostgresSaver(pool);  // Saves state to PostgreSQL
await checkpointer.setup();

const brain = workflow.compile({ checkpointer });

// ============================================
// STEP 6: Run Brain!
// ============================================

const result = await brain.invoke(
  {
    userMessage: "Build me a ride-sharing app",
    userId: "user_123",
    projectId: "proj_456",
    phase: 'intent',
  },
  {
    configurable: { thread_id: "proj_456" },  // Enables state persistence
  }
);
```

Visual Graph Structure

```
flowchart TD
    START(["▶ START"]) --> IC["📝 Intent Compiler"]

    IC --> CL

    subgraph CL["🔄 COUNCIL LOOP"]
        direction TB
        ARCH["🏗️ Architect<br/>Claude Opus 4.6"] --> ATK["⚔️ Attacker<br/>GPT-5.2"]
        ATK --> VER["✅ Verifier<br/>Qwen3 LOCAL"]
        VER -->|"Attacks remain? YES"| ARCH
    end

    CL -->|"No attacks remain"| MAD["🧩 MAD Decomposition"]

    MAD --> BL

    subgraph BL["🔨 BUILD LOOP"]
        direction TB
```

```
        BUILD["Build Micro-Task<br/>+ Firewall Check<br/>+ Run Tests"]
        BUILD -->|"✅ Pass"| NEXT["Next Task"]
        BUILD -->|"❌ Fail"| STRIKE["Strike 1: Root cause + retry<br/>Strike 2: Switch
        STRIKE --> BUILD
        NEXT --> BUILD
    end

    BL -->|"All tasks done"| VERI["🔬 Vericoding<br/>Dafny / Z3 Proofs"]
    VERI --> SURV["🛡️ Survival Test<br/>6 Dimensions"]
    SURV --> MEM["🧠 Update Memory<br/>3 Levels"]
    MEM --> ENDNODE(["   END"])
```

---

11. GRANDMASTER ROUTER — How Brain Picks the Right Model

```
// packages/router/src/router.ts

interface RoutingRequest {
  task: TaskType;           // 'architecture', 'code_gen', 'testing', 'verification'
  complexity: 1 | 2 | 3;   // 1=simple, 2=medium, 3=complex
  privacySensitive: boolean;
  maxCost: number;          // User's budget ceiling
  maxLatency: number;       // Milliseconds
}

interface ModelProfile {
  id: string;
  provider: string;
  costPer1MInput: number;
  costPer1MOutput: number;
  avgLatencyMs: number;
  accuracyByTask: Record<TaskType, number>;    // 0-100
  reliabilityScore: number;                     // From routing history
  isLocal: boolean;
}

const MODEL_PROFILES: ModelProfile[] = [
  {
    id: 'claude-opus-4-6',
    provider: 'anthropic',
    costPer1MInput: 15, costPer1MOutput: 75,
    avgLatencyMs: 8000,
    accuracyByTask: { architecture: 97, code_gen: 92, testing: 88, verification: 85 },
    reliabilityScore: 98,
    isLocal: false,
  },
  {
    id: 'gpt-5.3-codex',
    provider: 'openai',
    costPer1MInput: 2, costPer1MOutput: 8,
    avgLatencyMs: 3000,
```

```
    accuracyByTask: { architecture: 80, code_gen: 95, testing: 85, verification: 75 },
    reliabilityScore: 96,
    isLocal: false,
  },
  {
    id: 'qwen3-coder-next',
    provider: 'local',
    costPer1MInput: 0, costPer1MOutput: 0,
    avgLatencyMs: 5000,
    accuracyByTask: { architecture: 70, code_gen: 82, testing: 78, verification: 80 },
    reliabilityScore: 90,
    isLocal: true,
  },
  // ... more models
];

function selectModel(request: RoutingRequest): ModelProfile {
  const candidates = MODEL_PROFILES
    .filter(m => {
      // If privacy-sensitive, only local models
      if (request.privacySensitive && !m.isLocal) return false;
      // Must meet latency requirement
      if (m.avgLatencyMs > request.maxLatency) return false;
      return true;
    })
    .map(m => ({
      ...m,
      score: calculateScore(m, request),  // Weighted score
    }))
    .sort((a, b) => b.score - a.score);

  return candidates[0];
}

function calculateScore(model: ModelProfile, request: RoutingRequest): number {
  const accuracy = model.accuracyByTask[request.task] || 50;
  const costScore = 100 - (model.costPer1MOutput / 75 * 100);  // Normalize
  const reliability = model.reliabilityScore;

  // Weights shift based on task type:
  // Architecture: accuracy matters most
  // Verification: accuracy + local preferred
  // Testing: speed matters most
  const weights = TASK_WEIGHTS[request.task];

  return (
    accuracy * weights.accuracy +
    costScore * weights.cost +
    reliability * weights.reliability +
    (model.isLocal ? 10 : 0) * weights.privacy
  );
}
```

```
// ROUTING TABLE (what Brain actually uses):
//
| Task | Primary Model | Fallback Model |
|------|---------------|----------------|
| **Architecture Design** | Claude Opus 4.6 | GPT-5.2 |
| **Attack Generation** | GPT-5.2 | Claude Sonnet 4.5 |
| **Code Generation** | GPT-5.3-Codex | Claude Sonnet 4.5 |
| **Testing** | Gemini 3 Flash | GPT-5.3-Codex |
| **Verification** | Qwen3 (LOCAL) | Gemini 3 Flash |
| **Documentation** | Gemini 2.5 Pro | Claude Sonnet 4.5 |
| **Vision/UI** | Kimi K2.5 | GPT-5.2 |
| **Simple Tasks** | Qwen3/DeepSeek (LOCAL) | Claude Haiku 3.5 |
| **Explanations** | Claude Sonnet 4.5 | Gemini 2.5 Pro |
```

---

12. DATABASE SCHEMAS

PostgreSQL — Relational Data

```
-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  name TEXT,
  plan TEXT DEFAULT 'free',          -- free, pro, enterprise
  created_at TIMESTAMPTZ DEFAULT NOW(),
  settings JSONB DEFAULT '{}'        -- UI preferences, defaults
);

-- Projects
CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  name TEXT NOT NULL,
  specification JSONB,               -- Full project spec from Intent Compiler
  invariants JSONB,                  -- Formal invariants array
  architecture JSONB,                -- Verified architecture from Council
  status TEXT DEFAULT 'draft',       -- draft, building, testing, verified, deployed
  survival_score INTEGER,
  total_cost DECIMAL(10,4) DEFAULT 0,
  cost_ceiling DECIMAL(10,4),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Build Tasks (MAD decomposition)
CREATE TABLE build_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id),
  task_index INTEGER NOT NULL,
```

```sql
  description TEXT NOT NULL,
  status TEXT DEFAULT 'pending',     -- pending, building, verified, failed
  model_used TEXT,                   -- Which model generated this
  generated_code TEXT,
  test_code TEXT,
  confidence_score DECIMAL(5,2),
  verification_status TEXT,          -- verified, tested, unverified
  proof_data JSONB,                  -- Formal proof if available
  explanation TEXT,                  -- Anti-epistemic-debt
  cost DECIMAL(10,6),
  latency_ms INTEGER,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Council Debates (Audit Trail)
CREATE TABLE council_debates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id),
  cycle_number INTEGER,
  architect_proposal JSONB,
  attacker_attacks JSONB,
  verifier_results JSONB,
  proven_attacks INTEGER,
  resolved BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Governance Audit Log
CREATE TABLE audit_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id),
  user_id UUID REFERENCES users(id),
  action TEXT NOT NULL,              -- SPEC_APPROVED, CODE_GENERATED, DEPLOY_APPROVED
  actor TEXT NOT NULL,               -- 'brain', 'user', 'admin'
  details JSONB,
  blast_radius JSONB,               -- Estimated impact of this action
  approval_status TEXT,              -- pending, approved, rejected
  rollback_available BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Decision Log (Why Brain made each choice)
CREATE TABLE decision_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id),
  decision_type TEXT,                -- tech_choice, architecture, pattern, library
  question TEXT,                     -- "Should we use WebSockets or SSE?"
  chosen_option TEXT,                -- "WebSockets"
  reasoning TEXT,                    -- "Bidirectional needed for real-time tracking"
  alternatives JSONB,                -- [{option: "SSE", why_not: "One-directional only
  confidence DECIMAL(5,2),
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```sql
-- Tool Registry (Self-Evolving Tools)
CREATE TABLE tools (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  description TEXT,
  category TEXT,                      -- auth, database, api, ui, testing
  code TEXT NOT NULL,
  test_code TEXT,
  usage_count INTEGER DEFAULT 0,
  success_count INTEGER DEFAULT 0,
  success_rate DECIMAL(5,2) GENERATED ALWAYS AS
    (CASE WHEN usage_count > 0 THEN success_count::DECIMAL / usage_count * 100 ELSE 0 E
  is_promoted BOOLEAN DEFAULT FALSE, -- Promoted to built-in after 50 successes
  applicable_when JSONB,              -- Conditions when this tool applies
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Routing History (Grandmaster Router learning)
CREATE TABLE routing_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_type TEXT,
  model_used TEXT,
  success BOOLEAN,
  latency_ms INTEGER,
  cost DECIMAL(10,6),
  task_complexity INTEGER,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Qdrant — Vector Collections

```
Collection: "personal_memory"
├── Vectors: 1536 dimensions (OpenAI text-embedding-3-large)
├── Payload fields:
│   ├── user_id (keyword, index: true)
│   ├── category (keyword): "preference", "style", "pain_point", "history"
│   ├── content (text): The actual memory content
│   ├── project_id (keyword, optional)
│   └── created_at (datetime)
├── Example entries:
│   ├── "User prefers TypeScript over JavaScript" (pref)
│   ├── "User was frustrated with Firestore 1MB limit" (pain_point)
│   └── "User's React style: functional + custom hooks" (style)

Collection: "project_memory"
├── Vectors: 1536 dimensions
├── Payload fields:
│   ├── project_id (keyword, index: true)
```

```
│   ├── category: "decision", "bug", "failed_approach", "pattern"
│   ├── content (text)
│   └── tags (keyword[])
├── Example: "Chose PostgreSQL over MongoDB — need ACID for payments"

Collection: "collective_patterns"
├── Vectors: 1536 dimensions
├── Payload fields:
│   ├── category: "pattern", "anti_pattern", "failure_cascade", "tool"
│   ├── confidence (float): 0-100
│   ├── usage_count (integer)
│   ├── languages (keyword[]): ["typescript", "python"]
│   └── frameworks (keyword[]): ["react", "next.js"]
├── Example: "Stripe webhooks need idempotency keys" (confidence: 99.2%)

Collection: "failure_signatures"
├── Vectors: 1536 dimensions
├── Payload: error message, root cause, fix, prevention
├── Purpose: When Brain encounters an error, search this collection
│            to find if a similar error was seen before + its fix
```

Neo4j — Knowledge Graph Schema

```
// Node types
(:User {id, name, preferences})
(:Project {id, name, status, survivalScore})
(:Component {id, name, type, filePath})
(:Decision {id, question, chosen, reasoning})
(:Invariant {id, expression, status})    // verified, tested, unverified
(:Attack {id, description, proven})
(:Bug {id, description, rootCause, fix})
(:Tool {id, name, successRate})

// Relationship types
(:User)-[:OWNS]->(:Project)
(:Project)-[:HAS_COMPONENT]->(:Component)
(:Component)-[:DEPENDS_ON]->(:Component)        // Dependency graph!
(:Component)-[:IMPLEMENTS]->(:Invariant)
(:Project)-[:DECIDED]->(:Decision)
(:Decision)-[:LED_TO]->(:Decision)               // Decision chain
(:Attack)-[:TARGETS]->(:Component)
(:Bug)-[:FOUND_IN]->(:Component)
(:Bug)-[:FIXED_BY]->(:Decision)
(:Tool)-[:USED_IN]->(:Project)

// Example queries:
// "What depends on the payment module?"
MATCH (c:Component)-[:DEPENDS_ON*]->(payment:Component {name: 'PaymentService'})
RETURN c.name, c.filePath
```

```
// "What decisions led to the current architecture?"
MATCH path = (d1:Decision)-[:LED_TO*]->(d2:Decision)
WHERE d2.question CONTAINS 'database'
RETURN path

// "What bugs have affected auth components across all projects?"
MATCH (b:Bug)-[:FOUND_IN]->(c:Component)
WHERE c.type = 'auth'
RETURN b.description, b.rootCause, b.fix, count(*) as frequency
ORDER BY frequency DESC
```

---

13. MCP SERVER IMPLEMENTATION

```
// packages/mcp-servers/filesystem/src/index.ts

import { Server } from '@modelcontextprotocol/sdk/server';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio';

const server = new Server({
  name: 'brain-filesystem',
  version: '1.0.0',
}, {
  capabilities: {
    tools: {},
    resources: {},
  }
});

// Tool: Read a file
server.setRequestHandler('tools/call', async (request) => {
  if (request.params.name === 'read_file') {
    const { path } = request.params.arguments;
    const content = await fs.readFile(path, 'utf-8');
    return { content: [{ type: 'text', text: content }] };
  }

  if (request.params.name === 'write_file') {
    const { path, content } = request.params.arguments;
    // GOVERNANCE CHECK: Log this action
    await auditTrail.log({
      action: 'FILE_WRITE',
      path,
      blastRadius: await estimateBlastRadius(path),
    });
    await fs.writeFile(path, content);
    return { content: [{ type: 'text', text: `Written to ${path}` }] };
  }

  if (request.params.name === 'search_codebase') {
    const { query, directory } = request.params.arguments;
```

```
    const results = await ripgrep(query, directory);
    return { content: [{ type: 'text', text: JSON.stringify(results) }] };
  }
});

// Resource: Expose project structure
server.setRequestHandler('resources/read', async (request) => {
  if (request.params.uri.startsWith('brain://project/')) {
    const projectId = request.params.uri.split('/')[3];
    const structure = await getProjectStructure(projectId);
    return { contents: [{ uri: request.params.uri, text: structure }] };
  }
});

// Start server
const transport = new StdioServerTransport();
await server.connect(transport);

// Brain exposes these MCP servers:
// ├── brain-filesystem   (read/write/search files)
// ├── brain-terminal     (run shell commands in sandbox)
// ├── brain-git          (commit, branch, diff, log)
// ├── brain-database     (query PostgreSQL/Qdrant/Neo4j)
// ├── brain-browser      (navigate, screenshot, interact)
// └── brain-governance   (check approvals, log actions)
```

---

14. DEPLOYMENT ARCHITECTURE

```
# docker/docker-compose.yml

version: '3.9'

services:
  # ============ BRAIN CORE ============
  brain-api:
    build: ../apps/api
    ports: ['4000:4000', '4001:4001']   # REST + WebSocket
    environment:
      - DATABASE_URL=postgresql://brain:pw@postgres:5432/brain
      - REDIS_URL=redis://redis:6379
      - QDRANT_URL=http://qdrant:6333
      - NEO4J_URI=bolt://neo4j:7687
    depends_on: [postgres, redis, qdrant, neo4j]

  brain-web:
    build: ../apps/web
    ports: ['3000:3000']

  brain-worker:                          # Background job processor
    build: ../apps/api
```

```
    command: node dist/worker.js
    environment: *brain-env              # Same env as API

  # ============ DATABASES ============
  postgres:
    image: postgres:16
    volumes: ['pgdata:/var/lib/postgresql/data']
    environment:
      POSTGRES_DB: brain
      POSTGRES_PASSWORD: pw

  redis:
    image: redis:7-alpine
    volumes: ['redisdata:/data']

  qdrant:
    image: qdrant/qdrant:latest
    ports: ['6333:6333']
    volumes: ['qdrantdata:/qdrant/storage']

  neo4j:
    image: neo4j:5
    ports: ['7474:7474', '7687:7687']
    volumes: ['neo4jdata:/data']
    environment:
      NEO4J_AUTH: neo4j/brain_password

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    ports: ['9000:9000', '9001:9001']
    volumes: ['miniodata:/data']

  # ============ LOCAL AI ============
  ollama:
    image: ollama/ollama
    ports: ['11434:11434']
    volumes: ['ollama_models:/root/.ollama']
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]        # GPU access for local models

volumes:
  pgdata:
  redisdata:
  qdrantdata:
  neo4jdata:
  miniodata:
  ollama_models:
```

Production (Cloud) Architecture

```
graph TD
    CDN["☁️ CLOUDFLARE CDN<br/>DDoS protection · SSL · Caching"]
    CDN --> LB["⚖️ LOAD BALANCER<br/>nginx / AWS ALB"]

    LB --> API1["🖥️ Brain API<br/>Instance 1<br/>auto-scale"]
    LB --> API2["🖥️ Brain API<br/>Instance 2<br/>auto-scale"]

    API1 --> NET
    API2 --> NET

    subgraph NET["🔒 INTERNAL NETWORK"]
        PG["🐘 PostgreSQL<br/>Primary"]
        RD["⚡ Redis<br/>Cluster"]
        QD["🔍 Qdrant<br/>Cluster"]
        N4["🕸️ Neo4j<br/>Cluster"]
        GPU["🎮 GPU Inference Node<br/>SGLang + Qwen3 + DeepSeek V3.2"]
    end

    API1 -.->|"HTTPS"| EXT1["🟣 Anthropic API — Claude"]
    API1 -.->|"HTTPS"| EXT2["🟢 OpenAI API — GPT-5"]
    API1 -.->|"HTTPS"| EXT3["🔵 Google AI API — Gemini"]
    API1 -.->|"HTTPS"| EXT4["🟡 Moonshot API — Kimi"]
    API1 -.->|"HTTPS"| EXT5["📦 E2B API — Sandboxes"]
```

---

15. MONTHLY COST ESTIMATES

For a team of 5 developers using Brain daily:

Service	Monthly Cost
Anthropic (Claude) API	$200-500
OpenAI (GPT-5/Codex) API	$150-400
Google (Gemini) API	$30-80
Moonshot (Kimi) API	$20-50
E2B Sandbox	$50-100
LangSmith Tracing	$0-39
Cloud Hosting (API+DB)	$100-300
GPU Instance (local models)	$200-500
Storage (MinIO/S3)	$10-30
TOTAL (small team)	$760-2,000
TOTAL per developer/month	$152-400
Compared to:	
Devin ($500/seat/month)	$2,500

```
|   Copilot Enterprise ($39 ×5)| $195         |
|   Cursor Pro ($20 × 5)       | $100         |
|   Brain's VALUE (saved time) | 10-20 hrs/dev|
└────────────────────────────┴──────────────┘

Cost Optimization Strategies:
1. Route 60% of tasks to local models (Qwen3) = $0 inference
2. Use Gemini Flash for testing = $0.075/1M tokens
3. Cache with SGLang RadixAttention = 3-5x fewer tokens
4. Batch similar tasks = fewer API calls
5. User-set cost ceilings = never exceed budget
```

---

16. COMPLETE REQUEST LIFECYCLE — User Click to Code Output

What Happens When a User Types "Build me a todo app"

```
### Request Lifecycle Sequence

```mermaid
sequenceDiagram
    participant U as 🧑 User (UI)
    participant API as 🖥️ Brain API
    participant MEM as 🧠 Memory (Qdrant/Neo4j)
    participant LLM as 🤖 LLM (Claude/GPT)
    participant SBX as 📦 E2B Sandbox
    participant V as ✅ Verifier (Dafny/Z3)

    U->>API: POST /api/v1/projects/build (T+5ms)
    API->>MEM: Load Personal/Collective Memory (T+20ms)
    API->>LLM: Intent Compiler: Generate Questions (T+50ms)
    LLM-->>API: Clarification Questions (T+3000ms)
    API-->>U: WS: Send Questions (T+3005ms)
    U->>API: User Answers (T+30000ms)
    API->>LLM: Generate Spec + Invariants (T+30050ms)
    LLM-->>API: Specification JSON
    API-->>U: WS: Show Spec for Approval (T+33000ms)
    U->>API: User Approves (T+33050ms)

    rect rgb(200, 230, 255)
        Note over API, LLM: 🔄 COUNCIL PHASE
        API->>LLM: Propose Architecture (Opus)
        LLM-->>API: Proposal
        API->>LLM: Attack Design (GPT-5)
        LLM-->>API: Vulnerabilities Found
        API->>V: Prove Attacks (Qwen3 Local)
        V-->>API: Proven Attacks
        API->>LLM: Fix Attacks (Cycle 1-2)
```
```

```
    end

    rect rgb(230, 255, 200)
        Note over API, SBX: 🔨 BUILD PHASE (MAD Decomposition)
        API->>LLM: Decompose into Micro-Tasks
        loop Each Micro-Task
            API->>LLM: Generate Code + Tests
            API->>SBX: Run Tests in Sandbox
            SBX-->>API: Results (Pass/Fail)
            Note right of API: Firewall Check
        end
    end

    rect rgb(255, 230, 200)
        Note over API, V: 🔬 VERIFICATION & SURVIVAL
        API->>V: Dafny/Z3 Formal Proofs
        API->>SBX: Run 6D Survival Tests
    end

    API->>MEM: Update Memory (L1, L2, L3)
    API-->>U: WS: DONE! Download Project (T+510100ms)
```

Detailed Execution Trace (T+0 to T+510s)

Timestamp	Event	Details
T+0ms	**User Input**	User clicks SEND in Brain UI.
T+10ms	**API Entry**	Server verifies JWT, checks rate limits, and initializes LangGraph.
T+50ms	**Intent Logic**	Qdrant memory fetch for "user preferences" and "past patterns".
T+3000ms	**Claude Call**	First LLM response received with clarification questions.
T+30s	**User Input**	User answers questions via WebSocket.
T+33s	**Spec Ready**	Project specification (features, invariants) generated and approved.
T+33s+	**Council Phase**	**Architect (Claude Opus)** designs → **Attacker (GPT-5)** probes → **Verifier (Local)** confirms.
T+1m	**Build Phase**	**MAD Decomposition**: 15 micro-tasks identified.
T+1m - T+8m	**Build Loop**	15 iterations of: **Code Gen** → **Firewall** → **Test Gen** → **Sandbox Run**.
T+8m	**Verification**	Formal proofs for critical functions (add, delete, sync) via Dafny & Z3.
T+510s	**Completion**	**Survival Score: 91/100**. Memory updated. Project delivered.

```
    | | ✅ Todo App Built Successfully!          |
    | |                                          |
    | | 📊 Stats:                                |
    | | ├── 15 files, 1,200 lines of code        |
    | | ├── 89 tests, all passing                |
    | | ├── 4 functions formally VERIFIED        |
```

```
│ │ ├── Survival Score: 91/100                  │
│ │ ├── Average confidence: 93%                 │
│ │ ├── Total cost: $0.42                       │
│ │ └── Total time: 8.5 minutes                 │
│ │                                             │
│ │ 🔒 Security: OWASP clean                    │
│ │ 📝 Every function has an explanation        │
│ │ 🧪 Chaos tested: works offline              │
│ │                                             │
│ │ [View Code] [Download] [Deploy] [Explain]   │
│ └─────────────────────────────────────────────┘
│
```

T+510200ms │ Total time: ~8.5 minutes
│ Total cost: $0.42
│ API calls made: 28 (to 4 different providers + local)
│ LangSmith trace: https://smith.langchain.com/trace/xxx

---

17. BRAIN API ENDPOINTS — Every Route

Brain API (apps/api) — Full Endpoint List
Base URL: http://localhost:4000/api/v1

METHOD	ENDPOINT	PURPOSE
AUTH		
POST	/auth/register	Create new user account
POST	/auth/login	Get JWT token
POST	/auth/refresh	Refresh expired JWT
GET	/auth/me	Get current user profile
PROJECTS		
GET	/projects	List all user's projects
POST	/projects	Create new project
GET	/projects/:id	Get project details + status
DELETE	/projects/:id	Delete project
GET	/projects/:id/spec	Get project specification
GET	/projects/:id/architecture	Get verified architecture
GET	/projects/:id/files	List generated files
GET	/projects/:id/files/:fileId	Get file content + explain
BUILD (main workflow trigger)		
POST	/projects/:id/build	Start Brain workflow
GET	/projects/:id/build/status	Get current build status

```
╟─────────────
```

```
│ GET │ /cost/current-session │ Get current session cost │
│ POST │ /cost/ceiling │ Set spending limit │
├──────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ WEBSOCKET │
├──────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ WS │ ws://localhost:4001 │ Real-time build updates │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

---

18. EXAMPLE API REQUEST/RESPONSE FOR EVERY MAJOR ENDPOINT

3.1 POST /api/v1/projects/:id/build — Start a Build

REQUEST:
```
────────────────────────────────────────────────────────────────────────────────
POST /api/v1/projects/proj_abc123/build
Host: localhost:4000
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{
"message": "Build me a todo app with categories",
"options": {
"costCeiling": 5.00,
"preferLocal": true,
"autoApproveSpec": false,
"survivalThreshold": 80
}
}
```

RESPONSE:
```
────────────────────────────────────────────────────────────────────────────────
HTTP/1.1 202 Accepted
Content-Type: application/json

{
"buildId": "build_xyz789",
"projectId": "proj_abc123",
"status": "started",
"phase": "intent",
"websocketUrl": "ws://localhost:4001?buildId=build_xyz789",
"estimatedTime": "8-15 minutes",
"message": "Brain is starting. Connect to WebSocket for live updates."
}
```

3.2 WebSocket Messages (what the frontend receives in real-time)

```
// Phase updates
{ "type": "phase_change", "phase": "intent", "message": "Understanding your request..." }
```

{ "type": "phase_change", "phase": "council", "message": "Debating architecture..." }
{ "type": "phase_change", "phase": "build", "message": "Building micro-tasks..." }
{ "type": "phase_change", "phase": "verify", "message": "Verifying code proofs..." }
{ "type": "phase_change", "phase": "survival", "message": "Running survival tests..." }
{ "type": "phase_change", "phase": "complete", "message": "Done!" }

// Brain asks a question (user must answer)

{
"type": "question",
"questionId": "q_001",
"phase": "intent",
"questions": [
"Do you want user accounts or just local storage?",
"Do you need categories/tags for todos?",
"Should todos persist across browser sessions?"
],
"waitingForAnswer": true
}

// Build progress (per micro-task)

{
"type": "build_progress",
"taskIndex": 3,
"totalTasks": 15,
"taskName": "Create TodoContext",
"status": "completed",
"confidence": 94,
"firewallPassed": true,
"testsRun": 8,
"testsPassed": 8,
"costSoFar": 0.12
}

// Council debate (live)

{
"type": "council_update",
"cycle": 1,
"agent": "attacker",
"content": "Found vulnerability: localStorage 5MB limit",
"attackProven": true
}

// Cost update

{ "type": "cost_update", "current": 0.32, "ceiling": 5.00, "remaining": 4.68 }

// Error / Strike

{
"type": "strike",
"strikeNumber": 1,
"taskName": "Create storage service",
"error": "TypeError: localStorage.setItem is not a function",

"rootCause": "Running in Node.js environment without browser APIs",
"fix": "Switching to mock storage for testing, real localStorage in browser"
}

// Final result
{
"type": "complete",
"result": {
"filesGenerated": 15,
"totalLines": 1200,
"testsTotal": 89,
"testsPassing": 89,
"verifiedFunctions": 4,
"survivalScore": 91,
"avgConfidence": 93,
"totalCost": 0.42,
"totalTime": "8m 30s"
}
}

3.3 GET /api/v1/projects/:id/survival — Get Survival Report

REQUEST:

GET /api/v1/projects/proj_abc123/survival
Authorization: Bearer eyJhbGciOiJIUzI1Nils...

RESPONSE:

{
"projectId": "proj_abc123",
"overallScore": 91,
"verdict": "PRODUCTION_READY",
"runAt": "2026-02-16T05:35:00Z",
"dimensions": {
"dependencies": {
"score": 95,
"details": {
"totalPackages": 8,
"vulnerabilities": 0,
"outdated": 1,
"unmaintained": 0,
"autoPatched": 0
}
},
"security": {
"score": 98,
"details": {
"sqlInjection": "SAFE",
"xss": "SAFE",

"authBypass": "SAFE",
"exposedSecrets": "SAFE",
"insecureDeserialization": "SAFE"
}
},
"scale": {
"score": 90,
"details": {
"10_users_ms": 5,
"100_users_ms": 12,
"1000_users_ms": 45,
"10000_todos_render_ms": 89,
"bottleneck": "Array filter on large dataset",
"autoFix": "Added useMemo for filtered todos",
"afterFix_ms": 23
}
},
"chaos": {
"score": 88,
"details": {
"offlineMode": "WORKS — data persists in localStorage",
"storageQuotaExceeded": "HANDLED — shows warning at 80% capacity",
"corruptData": "HANDLED — JSON parse wrapped in try/catch",
"coldStart": "1.2 seconds"
}
},
"maintainability": {
"score": 91,
"details": {
"codeDuplication": "1.8%",
"avgFunctionLength": 14,
"moduleCoupling": "LOW",
"readabilityScore": 89,
"functionsWithExplanations": "100%"
}
},
"futureProofing": {
"score": 85,
"details": {
"canAddBackend": true,
"canAddAuth": true,
"canMigrateDB": true,
"nodeCompatibility": "22, 23 tested",
"recommendation": "Abstract storage layer for easy DB migration"
}
}
}
}

---

19. HOW MEMORY CORTEX STORES AND RETRIEVES DATA

Writing to Memory (after a project is complete)

STEP 1: Generate embedding vector from the content

```
// Using OpenAI embeddings API
POST https://api.openai.com/v1/embeddings
{
"model": "text-embedding-3-large",
"input": "User prefers TypeScript, React with custom hooks, Tailwind CSS",
"dimensions": 1536
}
```

Response: { "embedding": [0.023, -0.041, 0.087, ...1536 numbers...] }

STEP 2: Store in Qdrant (vector database)

```
PUT http://localhost:6333/collections/personal_memory/points
{
"points": [{
"id": "mem_123",
"vector": [0.023, -0.041, 0.087, ...],
"payload": {
"user_id": "user_xyz",
"category": "preference",
"content": "User prefers TypeScript, React with custom hooks, Tailwind CSS",
"confidence": 0.95,
"source_project": "proj_abc123",
"created_at": "2026-02-16T05:35:00Z"
}
}]
}
```

STEP 3: Store relationships in Neo4j (knowledge graph)

```
// Create decision chain
MERGE (p:Project {id: "proj_abc123"})
MERGE (d1:Decision {
id: "dec_001",
question: "State management library?",
chosen: "React Context + useReducer",
reasoning: "Simple app, no need for Redux/Zustand"
})
MERGE (d2:Decision {
id: "dec_002",
question: "Storage approach?",
chosen: "localStorage with abstraction layer",
reasoning: "User wants persistence, no backend needed"
```

```
})
MERGE (p)-[:DECIDED]->(d1)
MERGE (p)-[:DECIDED]->(d2)
MERGE (d1)-[:LED_TO]->(d2)


### Reading from Memory (at the start of a new project)
```

STEP 1: User says "Build me a shopping cart"

STEP 2: Brain searches Personal Memory (Qdrant)

POST http://localhost:6333/collections/personal_memory/points/search
{
"vector": [embedding of "shopping cart app"],
"filter": { "must": [{ "key": "user_id", "match": { "value": "user_xyz" }}]},
"limit": 10,
"with_payload": true
}

Response:
[
{ "score": 0.89, "payload": { "content": "User prefers TypeScript + React" }},
{ "score": 0.82, "payload": { "content": "User frustrated with Firestore 1MB limit" }},
{ "score": 0.76, "payload": { "content": "User's last project used Stripe for payments" }}
]

STEP 3: Brain searches Collective Memory (Qdrant)

POST http://localhost:6333/collections/collective_patterns/points/search
{
"vector": [embedding of "shopping cart ecommerce"],
"filter": { "must": [{ "key": "languages", "match": { "any": ["typescript"] }}]},
"limit": 10
}

Response:
[
{ "score": 0.94, "payload": {
"content": "Stripe webhooks need idempotency keys",
"confidence": 99.2, "usage_count": 847
}},
{ "score": 0.91, "payload": {
"content": "Cart state: use optimistic updates + server reconciliation",
"confidence": 87.5, "usage_count": 234
}},
{ "score": 0.88, "payload": {
"content": "ANTI-PATTERN: Don't store prices in cart — always fetch from server",
"confidence": 95.1, "usage_count": 156
}}
]

STEP 4: Brain searches Failure Signatures (Qdrant)

POST http://localhost:6333/collections/failure_signatures/points/search
{
"vector": [embedding of "shopping cart checkout payment"],
"limit": 5
}

Response:
[
{ "score": 0.92, "payload": {
"error": "Stripe checkout → webhook fails → order stuck in pending",
"root_cause": "Missing webhook retry handler",
"fix": "Add exponential backoff + idempotency check",
"prevention": "Always implement webhook retry from day 1"
}}
]

STEP 5: Brain now KNOWS before writing any code:
├── User wants TypeScript + React
├── User had issues with Firestore limits → suggest PostgreSQL
├── User used Stripe before → include Stripe integration
├── Must add idempotency keys to webhooks (99.2% confidence pattern)
├── Use optimistic updates for cart (87.5% confidence)
├── Never store prices in cart (95.1% confidence anti-pattern)
└── Pre-emptively handle webhook failures (known failure cascade)

---

20. HOW THE GRANDMASTER ROUTER LEARNS OVER TIME

INITIAL STATE (Day 1):

Router uses DEFAULT weights:
Architecture → Claude Opus (accuracy: 97)
Code gen → GPT-5.3-Codex (accuracy: 95)
Testing → Gemini Flash (cost: cheapest)
Verification → Qwen3 Local (cost: $0)

AFTER 100 PROJECTS (Day 30):

Router queries routing_history table:

SELECT model_used, task_type,
AVG(CASE WHEN success THEN 100 ELSE 0 END) as success_rate,
AVG(latency_ms) as avg_latency,
AVG(cost) as avg_cost
FROM routing_history

GROUP BY model_used, task_type
ORDER BY success_rate DESC

Results might show:

Model	Task	Success	Latency	Cost	
Claude Opus	architecture	98%	8200ms	$0.08	
GPT-5.3-Codex	code_gen	94%	2800ms	$0.003	
DeepSeek V3.2	code_gen	92%	4200ms	$0.00	← Surprise!
Qwen3 Local	verification	89%	5100ms	$0.00	
Gemini Flash	testing	91%	1200ms	$0.001	
Gemini Flash	code_gen	78%	1100ms	$0.001	← Bad at code

Router LEARNS:
✅ DeepSeek V3.2 is almost as good as Codex for code gen, and FREE
✅ Gemini Flash is terrible at code gen → never route code there
✅ For simple code tasks, use DeepSeek locally → save $0.003 per task

NEW routing table (auto-updated):
Simple code (complexity 1) → DeepSeek V3.2 LOCAL ($0.00)
Medium code (complexity 2) → GPT-5.3-Codex ($0.003)
Complex code (complexity 3) → Claude Sonnet ($0.01)
Architecture → Claude Opus ($0.08) [unchanged, still best]
Testing → Gemini Flash ($0.001) [unchanged, still cheapest]

---

21. HOW MCP CONNECTS BRAIN TO THE WORLD

```mermaid
graph TD
    subgraph BRAIN_MCP["🧠 Brain-Hosted MCP Servers (Inside-Out)"]
        FS["📁 Filesystem<br/>read_file · write_file<br/>search · list_dir"]
        TERM["💻 Terminal<br/>run_command · run_tests<br/>install_pkg"]
        GIT["🌲 Git<br/>commit · branch<br/>diff · push"]
        DB["🗄️ Database<br/>query_pg · search_qdrant<br/>query_neo4j"]
        BROW["🌐 Browser<br/>navigate · screenshot<br/>click · type"]
        GOV["⚖️ Governance<br/>log_action · check_approval<br/>rollback"]
    end

    subgraph EXT_MCP["🔌 External MCP Hub (Outside-In)"]
        GH["🐙 GitHub<br/>create_pr · review_pr · merge"]
        JR["🎫 Jira<br/>create_issue · update_status"]
        SL["💬 Slack<br/>send_message · notify_team"]
        FG["🎨 Figma<br/>get_design · export_svg"]
        VC["🚀 Vercel<br/>deploy · env_vars"]
        ST["💳 Stripe<br/>test_webhook · create_key"]
    end
```

```
BRAIN_MCP <==>|"Model Context Protocol"| EXT_MCP
```

Example Multi-Server Flow

1. **Brain (Build)**: Finishes generating and testing code in isolated sandbox.
2. **Brain → Git MCP**: Commits code and pushes to a fresh feature branch.
3. **Brain → GitHub MCP**: Creates a Pull Request with the **Survival Report** as the description.
4. **Brain → Slack MCP**: Notifies the dev team channel with the PR link and confidence score.

---

22. ERROR HANDLING & RECOVERY FLOWCHART

```
flowchart TD
    ERR["🚨 ERROR OCCURS"] --> PHASE{"Which phase?"}

    PHASE -->|"LLM API"| LLM_ERR["LLM API Error"]
    PHASE -->|"Build"| BUILD_ERR["Build Error"]
    PHASE -->|"Test/Verify"| TEST_ERR["Test/Verify Error"]

    LLM_ERR --> LLM_TYPES{"Error Type?"}
    LLM_TYPES -->|"Rate Limit"| WAIT["Wait + Exponential Backoff"]
    LLM_TYPES -->|"Auth Error"| REFRESH["Refresh API Key"]
    LLM_TYPES -->|"Model Down"| FALLBACK["Switch to Fallback Model"]
    LLM_TYPES -->|"Timeout"| FASTER["Try Smaller/Faster Model"]
    LLM_TYPES -->|"Budget Hit"| PAUSE["Pause + Ask User"]

    BUILD_ERR --> STRIKE["3-STRIKE PROTOCOL"]
    subgraph STRIKE
        S1["Strike 1: Root cause analysis + retry"]
        S2["Strike 2: Switch model + different approach"]
        S3["Strike 3: Admit stuck, ask human for help"]
        S1 --> S2 --> S3
    end

    TEST_ERR --> AUTO["Auto-fix + Re-run"]
    AUTO -->|"Fix fails"| HUMAN["Flag for human review"]
```

Logging Infrastructure:

- **PostgreSQL**: `audit_log` table (permanent record).
- **LangSmith**: Execution traces with granular error details.
- **Redis**: Real-time status updates for UI notifications.
- **Qdrant**: `failure_signatures` (Brain learns from mistakes).
- **WebSocket**: Immediate UI feedback to the user.

---

23. SECURITY ARCHITECTURE

Request Security Flow

```
flowchart TD
    UB["🧑 User Browser"] -->|"HTTPS (TLS 1.3)"| API["🖥️ Brain API"]

    subgraph MIDDLEWARE["🛡️ Security Middleware"]
        RL["⚖️ Rate Limiter (Redis)<br/>100 req/min"]
        JWT["🔑 JWT Verify (RS256)<br/>Expiry & Scopes"]
        SAN["🧼 Input Sanitize<br/>XSS / SQLi / Size"]
        ADT["📝 Audit Logger<br/>WHO / WHAT / WHEN"]
        RL --> JWT --> SAN --> ADT
    end

    API --> RL
    ADT --> BIZ["🚀 Route Handler<br/>(Business Logic)"]
```

Data Security Matrix

Component	Protection	Notes
User Code	AES-256 Encryption at rest	Encrypted in PostgreSQL
API Keys	Environment Variables	NEVER stored in DB or code
Personal Memory (L1)	User-level ACL + Encryption	Owner-only access
Collective Memory (L3)	Anonymized Pattern Extraction	No source code shared
Local Inference	0-Data Retention (On-Device)	Code never leaves machine
Sandboxes	Ephemeral E2B Instances	Destroyed & isolated
Backups	Versioned & Encrypted	Stored in S3/MinIO
Compliance	GDPR "Right to be Forgotten"	Data deletion on request

---